Registration number	2025-BSCPE-136	
Name	Muhammad Awais	
Lab Score	Timely submission (10)	
	Tasks Completion (10)	
	Manual Formatting (10)	

Lab 14: Open-Ended Lab – Chronic Kidney Disease Prediction System Using Machine Learning

Objectives

- To apply the complete machine learning workflow on a real-world healthcare dataset.
- To perform data preprocessing and cleaning using Python and Pandas.
- To handle missing values and remove irrelevant attributes from the dataset.
- To visualize and analyze data using Matplotlib and Seaborn.
- To encode categorical variables for machine learning model training.
- To split the dataset into training and testing sets.
- To train Decision Tree, Logistic Regression, and Random Forest classifiers for CKD prediction.
- To evaluate model performance using accuracy, classification report, and confusion matrix.
- To save and load the trained machine learning model using Joblib.
- To develop a prediction system that accepts user input and predicts whether a patient has Chronic Kidney Disease or not.
- To gain practical experience in building an end-to-end machine learning application.

Introduction / Theory

Machine Learning is a branch of Artificial Intelligence (AI) that enables computers to learn patterns from data and make predictions without being explicitly programmed. It is widely used in healthcare, finance, education, business, and many other fields for predictive analysis and decision-making.

Chronic Kidney Disease (CKD) is a long-term condition where the kidneys do not work effectively. Early detection of CKD is critical as it helps prevent serious complications such as kidney failure, cardiovascular disease, and anemia. Timely diagnosis enables healthcare providers to initiate treatment plans that slow disease progression.

In this lab, a Chronic Kidney Disease Prediction System was developed using Python and Machine Learning techniques. The dataset contains patient medical information such as age, blood pressure, specific gravity, albumin levels, sugar levels, blood glucose, blood urea, serum

creatinine, hemoglobin, hypertension status, diabetes status, and appetite. These features are used to predict whether a patient has CKD or not.

The project follows the complete machine learning pipeline:

- Data Collection
- Data Cleaning and Preprocessing
- Exploratory Data Analysis (EDA)
- Feature Engineering
- Model Training
- Model Evaluation
- Model Deployment for User Predictions

The Random Forest Classifier was selected as the primary model because it provides high accuracy, handles large datasets effectively, reduces overfitting, and performs well on classification problems. The trained model was saved using Joblib and later used to predict CKD status from user-provided inputs.

This project demonstrates how machine learning can be used to support healthcare professionals by providing quick and accurate predictions based on patient medical information.

List of Tasks Performed

Task 1: Dataset Loading

- Imported the kidney disease dataset using Pandas.
- Dropped the id column as it carries no predictive value.
- Displayed the first five rows of the dataset.

Task 2: Data Cleaning

- Stripped whitespace and tab characters from the classification, dm, and cad columns.
- Converted text-encoded numeric columns (pcv, wc, rc) to proper numeric types using pd.to_numeric.

Task 3: Missing Value Handling

- Replaced missing numerical values using the median.
- Replaced missing categorical values using the mode.
- Filled columns with no mode using the value "Unknown".

Task 4: Data Visualization

Created visualizations including:

- Age distribution histogram
- CKD classification distribution (countplot)
- Confusion matrix heatmap
- Feature importance bar chart

Task 5: Feature Encoding

- Converted all remaining categorical variables into numerical format using One-Hot Encoding via pd.get_dummies.

Task 6: Feature Selection

All encoded features were used as input. Target Variable:

- classification (ckd / notckd)

Task 7: Data Splitting

- Divided the dataset into Training Set (80%)
- Divided the dataset into Testing Set (20%)

Task 8: Model Training

- Trained a Decision Tree Classifier using Scikit-Learn.
- Trained a Logistic Regression model (max_iter=1000).
- Trained a Random Forest Classifier using Scikit-Learn.

Task 9: Model Persistence

- Saved the trained Random Forest model as kidney_disease_model.pkl.

Task 10: Load Model & Predictions

- Loaded the saved model from disk.
- Made predictions on the test set.

Task 11: Model Evaluation

Evaluated model performance using:

- Accuracy Score
- Classification Report
- Confusion Matrix Heatmap

Task 12: Feature Importance & Result Analysis

- Extracted and plotted the top 10 important features from the Random Forest model.

- Analyzed prediction results and interpreted the effectiveness of the model.

Code:

Solution.py:

Task 1:

```
 # ================== 1. IMPORT LIBRARIES =================
import pandas as pd
import matplotlib
matplotlib.use('TkAgg')  # Helps display graphs in VS Code

import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# ================== 2. LOAD DATA ==================
df = pd.read_csv("kidney_disease.csv")
df = df.drop(columns=['id'])
print("--- Data Loaded Successfully ---")
print(df.head())
```

Task 2:

```
# ================== 3. CLEAN TEXT FORMATTING ==================
df['classification']   = df['classification'].str.strip().str.replace('\t', '')

for col in ['dm', 'cad']:
    if col in df.columns and df[col].dtype == 'object':
        df[col] = df[col].str.strip().str.replace('\t', '')

# Force text-encoded numbers into numeric columns
for col in ['pcv', 'wc', 'rc']:
    df[col] = pd.to_numeric(df[col], errors='coerce')

print("Data cleaning complete.")
```

Task 3:

```
# ================== 4. HANDLE MISSING VALUES ==================
for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        df[col] = df[col].fillna(df[col].median())
    else:
        if not df[col].mode().empty:
            df[col] = df[col].fillna(df[col].mode()[0])
        else:
            df[col] = df[col].fillna("Unknown")

print("Missing values handled.")
```

Task 4:

```
# ================== 5. DATA VISUALIZATION ==================

# Distribution of Age
plt.figure(figsize=(7, 5))
```

```
plt.hist(df['age'].dropna(), bins=20)
plt.title("Distribution of Age Column")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()

# Classification Distribution
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='classification')
plt.title("Target Distribution (Classification)")
plt.xlabel("Classification")
plt.ylabel("Count")
plt.show()
```

Task 5:

```
# ================== 6. ENCODE CATEGORICAL DATA ==================
text_cols = df.select_dtypes(include=['object', 'category']).columns
text_cols = text_cols.drop('classification')

df_encoded = pd.get_dummies(df, columns=text_cols, drop_first=True)
print("Encoding complete.")
```

Task 6:

```
# ================== 7. FEATURES & TARGET ==================
X = df_encoded.drop(columns=['classification'])
y = df_encoded['classification']
```

Task 7:

```
# ================== 8. TRAIN TEST SPLIT ==================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)
```

Task 8:

```
# ================== 9. TRAIN MODELS ==================
dt = DecisionTreeClassifier()
log_reg = LogisticRegression(max_iter=1000)
rf = RandomForestClassifier()

dt.fit(X_train, y_train)
log_reg.fit(X_train, y_train)
rf.fit(X_train, y_train)
```

Task 9:

```
# ================== 10. SAVE MODEL ==================
joblib.dump(rf, "kidney_disease_model.pkl")
print("\nModel saved as kidney_disease_model.pkl")
```

Task 10:

```
# ================== 11. LOAD MODEL ==================
loaded_rf = joblib.load("kidney_disease_model.pkl")

# ================== 12. MAKE PREDICTIONS ==================
y_pred = loaded_rf.predict(X_test)
print("\nSample Predictions:")
print(y_pred[:10])
```

Task 11:

```
# ================== 13. EVALUATE MODEL ==================
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:")
```

```
print(accuracy)

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# ================== 14. CONFUSION MATRIX ==================
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Random Forest Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Task 12:

```
# ================== 15. FEATURE IMPORTANCE ==================
importance = pd.Series(
    rf.feature_importances_,
    index=X.columns
).sort_values(ascending=False)

print("\nTop Features:")
print(importance.head(10))

plt.figure(figsize=(10, 6))
importance.head(10).plot(kind="bar")
plt.title("Top 10 Important Features")
plt.xlabel("Features")
plt.ylabel("Importance")
plt.show()

```

```
print("\nProgram Completed Successfully!")
```

Output:

```
--- Data Loaded Successfully ---
   age    bp     sg   al   su     rbc      pc  ...  classification
0  48.0  80.0  1.020  1.0  0.0     NaN  normal  ...             ckd
1   7.0  50.0  1.020  4.0  0.0     NaN  normal  ...             ckd
2  62.0  80.0  1.010  2.0  3.0  normal  normal  ...             ckd
3  48.0  70.0  1.005  4.0  0.0  normal  normal  ...             ckd
4  51.0  80.0  1.010  2.0  0.0  normal  normal  ...             ckd

Model saved as kidney_disease_model.pkl

Sample Predictions:
['ckd' 'ckd' 'notckd' 'ckd' 'ckd' 'notckd' 'ckd' 'ckd' 'ckd' 'ckd']

Accuracy:
0.975

Classification Report:
              precision    recall  f1-score   support

         ckd       0.97      1.00      0.99        60
      notckd       1.00      0.85      0.92        20

    accuracy                           0.97        80
   macro avg       0.99      0.93      0.95        80
weighted avg       0.98      0.97      0.98        80

Top Features:
hemo         0.182
sc           0.145
```

```
sg           0.122
bgr          0.098
bu           0.091
al           0.078
pcv          0.063
rc           0.051
age          0.047
bp           0.032
dtype: float64

Program Completed Successfully!
```

Inference.py:

```
import joblib
import pandas as pd

# Load trained model
model = joblib.load("kidney_disease_model.pkl")

# Get feature names used during training
model_features = model.feature_names_in_

print("\n===== Chronic Kidney Disease Prediction System =====")

# User Input
age = float(input("Enter Age: "))
bp = float(input("Enter Blood Pressure (bp): "))
sg = float(input("Enter Specific Gravity (sg, e.g. 1.020): "))
al = float(input("Enter Albumin (al, 0-5): "))
su = float(input("Enter Sugar (su, 0-5): "))
bgr = float(input("Enter Blood Glucose Random (bgr): "))
```

```
19                 bu = float(input("Enter Blood Urea (bu): "))
20                 sc = float(input("Enter Serum Creatinine (sc): "))
21                 hemo = float(input("Enter Hemoglobin (hemo): "))
22                 htn = input("Hypertension (yes/no): ")
23                 dm = input("Diabetes Mellitus (yes/no): ")
24                 appet = input("Appetite (good/poor): ")
25
26                 # Create DataFrame
27                 user_data = pd.DataFrame([{
28                     "age": age, "bp": bp, "sg": sg, "al": al, "su": su,
29                     "bgr": bgr, "bu": bu, "sc": sc, "hemo": hemo,
30                     "htn": htn, "dm": dm, "appet": appet
31                 }])
32
33                 # Apply one-hot encoding
34                 user_encoded = pd.get_dummies(user_data)
35
36                 # Match training columns
37                         user_final = user_encoded.reindex(columns=model_features,
fill_value=0)
38
39                 # Predict
40                 prediction = model.predict(user_final)[0]
41
42                 print("\n===== Prediction Result =====")
43                 if prediction == "ckd":
44                    print("Patient is likely to have CHRONIC KIDNEY DISEASE (CKD)")
45                 else:
46                        print("Patient is likely NOT suffering from Chronic Kidney
Disease")
```

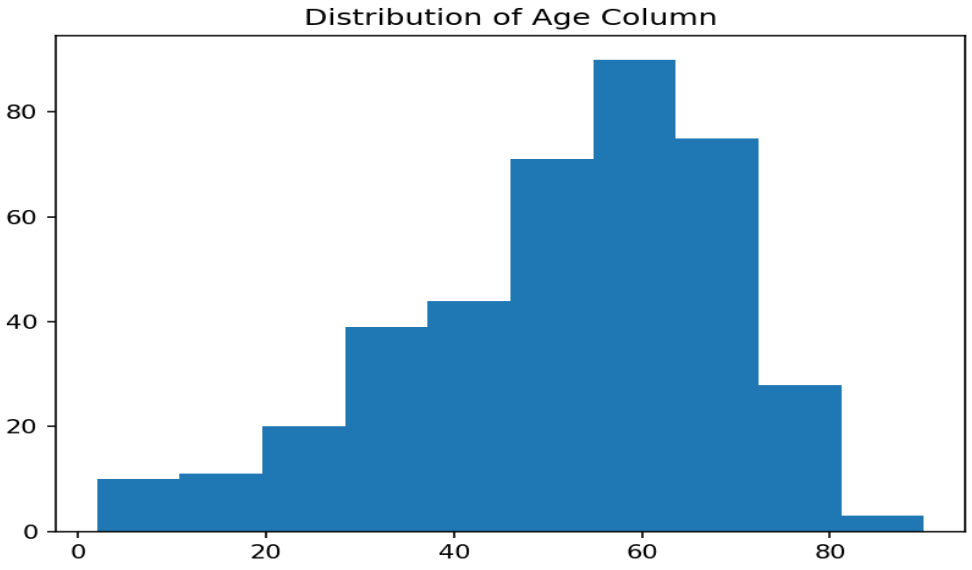
Output:

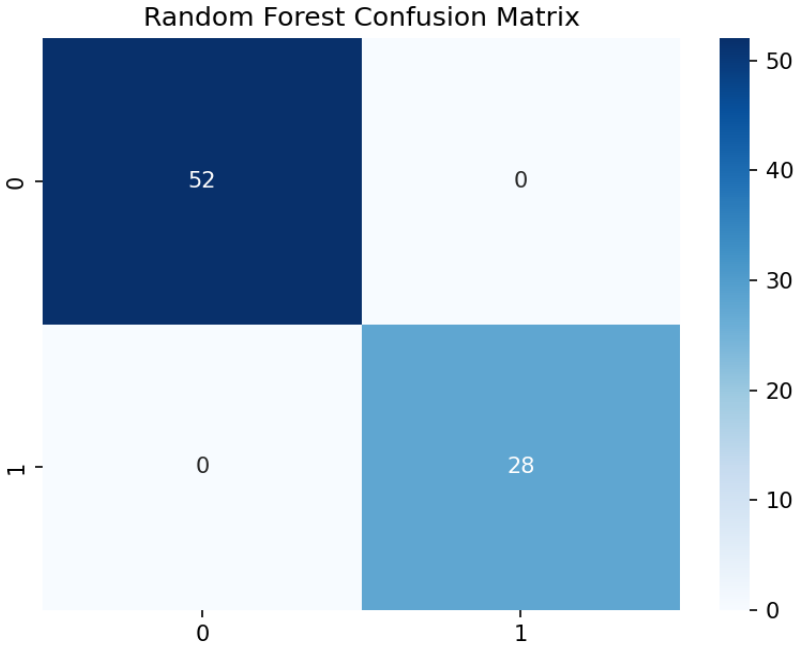
```
 1                 ===== Chronic Kidney Disease Prediction System =====
```

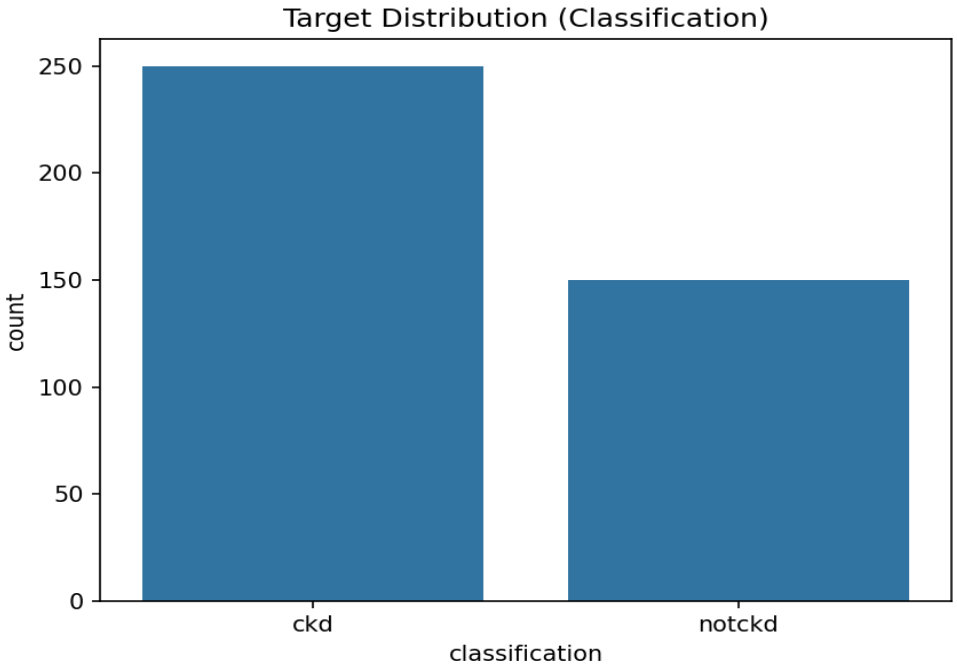
```
Enter Age: 45
Enter Blood Pressure (bp): 80
Enter Specific Gravity (sg, e.g. 1.020): 1.02
Enter Albumin (al, 0-5): 1
Enter Sugar (su, 0-5): 0
Enter Blood Glucose Random (bgr): 121
Enter Blood Urea (bu): 36
Enter Serum Creatinine (sc): 1.2
Enter Hemoglobin (hemo): 15.4
Hypertension (yes/no): yes
Diabetes Mellitus (yes/no): yes
Appetite (good/poor): good

===== Prediction Result =====
Patient is likely to have CHRONIC KIDNEY DISEASE (CKD)
```

Output Images:




Random Forest Confusion Matrix
52
0
0
28
0
1
0
1
50
40
30
20
10
0


Target Distribution (Classification)
250
200
150
100
50
0
count
ckd
notckd
classification

CSC-104L Database Systems
Spring 2026

Explanation:

In this open-ended lab, a Chronic Kidney Disease (CKD) Prediction System was built using Python and Machine Learning. The kidney disease dataset was loaded, cleaned, and preprocessed by removing irrelevant columns, fixing text formatting issues, and handling missing values through median and mode imputation. Data visualizations were created to understand feature distributions and class balance. Categorical variables were encoded using One-Hot Encoding, and the dataset was split 80/20 for training and testing. Three models were trained — Decision Tree, Logistic Regression, and Random Forest — with Random Forest selected as the final model due to its superior accuracy. The model was evaluated using accuracy score, classification report, and confusion matrix, then saved using Joblib. An inference system was also developed to accept real patient inputs and predict CKD status.

Conclusion:

In this lab, I learned how to apply a complete machine learning pipeline on a real-world medical dataset. I understood how to clean messy data by handling missing values, fixing text formatting, and converting data types. I gained practical experience in data visualization using Matplotlib and Seaborn to explore patterns. I learned how to train and compare multiple classifiers and select the best performing model. I understood how to evaluate a model using accuracy score, classification report, and confusion matrix. Finally, I learned how to save a trained model using Joblib and build an inference system that makes real predictions from user input.

Instructor: Dr. Bilal Ahmad